

that the fix can ship independently of any other work.

4.4 GPS-to-Street Matching Engine

The central technical challenge of Street Keeper was converting raw GPS coordinates from a user’s run into a determination of which streets were covered and by how much. Two matching engines were designed and implemented, referred to as V1 and V2, each using a fundamentally different approach to this problem.

4.4.1 Engine V1: Polyline Coverage

The V1 engine treated street matching as a geometric coverage problem. Given a GPS trace it computed a bounding box, queried the Overpass API for candidate streets, optionally snapped the trace using Mapbox Map Matching, and calculated what percentage of each street’s polyline was covered by the trace. Results were stored in `UserStreet-Progress` under a MAX rule, so that a user’s progress on any individual street could only increase across runs and a subsequent partial run could never regress an earlier full one. With Mapbox snapping enabled, matching accuracy measured on test activities sat at roughly 98%; without it, accuracy dropped to roughly 85% because raw GPS samples often landed a few metres off the true road centreline. Two limitations of this design were significant. The engine required an Overpass query for every activity, which tied the success of a run’s processing to the availability of a public third-party service; Overpass had observed downtime of several minutes at a time during development and there was no local fallback. The continuous-percentage model was also sensitive to GPS drift where two parallel streets a short distance apart could each accumulate partial coverage from a single run, effectively stealing credit from each other (Merry & Bettinger, 2019; Zandbergen, 2009). The V1 pipeline and the architectural weaknesses that motivated the migration are illustrated (see Figure 4.6 “The retired V1 matching engine, retained to motivate the V2 migration”, p.74).

4.4.2 Engine V2: Node Proximity

The V2 engine adopted a node-based model inspired by CityStrides, an established street-completion tracking platform (Chevalier, 2025). Rather than measuring polyline coverage, V2 treated the street network as a graph of nodes (junctions and intermediate waypoints on a way’s geometry). If a GPS point fell within 25 metres of a node (`snapRadiusM: 25` in `NODE_PROXIMITY_CONFIG`), that node was recorded as “hit” in the `UserNode-Hit` table. A street was considered complete when at least 90% of its nodes had been hit (`standardCompletionThreshold: 0.9`), with a stricter 100% threshold applied to short streets with ten or fewer nodes (`shortStreetNodeThreshold: 10`) so that a four-node cul-de-sac could not be marked complete after a single nearby point. Completion was therefore derived at query time from the ratio of hit nodes to total nodes per way rather than stored as a pre-computed percentage; this avoided a denormalised field

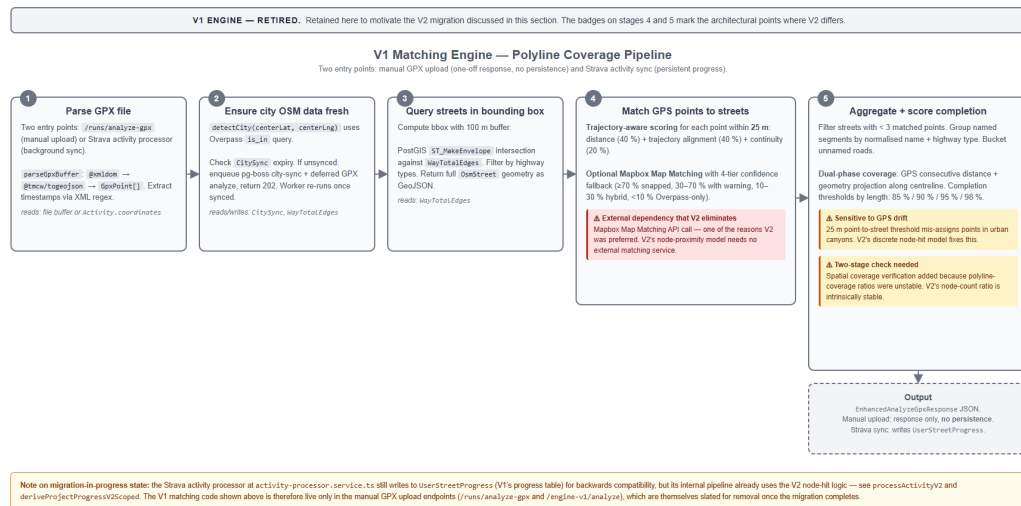


Figure 4.6: The retired V1 matching engine, retained here to motivate the V2 migration described in this section. V1 took GPS points and matched them to OSM streets via trajectory-aware scoring with an optional Mapbox Map Matching layer, then computed coverage ratios with two-stage spatial verification. The architectural weaknesses motivating the migration are flagged on stages 4 and 5: external dependency on Mapbox when configured, sensitivity to GPS drift in dense urban environments, and the need for a secondary spatial check because polyline-coverage ratios were unstable. V2's discrete node-proximity model (Figure 4.7) addresses all three.

that could drift out of sync with the authoritative hit data. This pipeline is illustrated (see Figure 4.7 “V2 node-proximity processing pipeline”, p.75).

V2 offered three advantages over V1. Per-activity Overpass calls were eliminated because node data was cached through city synchronisation, so processing a run depended only on the local database once a city had been synced once. The discrete node-hit model was deterministic, which removed the ambiguity of continuous percentage estimates and meant that a rerun of the same processing pipeline on the same GPS data produced the same result bit-for-bit. The approach also aligned with the CityStrides model users coming from that platform already understood, giving the “which streets have I completed?” question a familiar semantics.

The path to V2 went through an abandoned intermediate design. Early project documentation proposed a graph-based approach using Hidden Markov Model map matching to snap GPS traces to street edges, modelled as a Viterbi decoding over road segments. That design was abandoned before any matching code shipped because it depended on pre-seeding the database from a PBF extract, the same pre-seed path that the Supabase storage limit ultimately invalidated (see subsection 5.6.3 “Database Storage Limit and the City Sync Pivot”, p.98). The pivot to node proximity with on-demand city synchronisation preserved the benefits of having map data local to the application (no per-activity

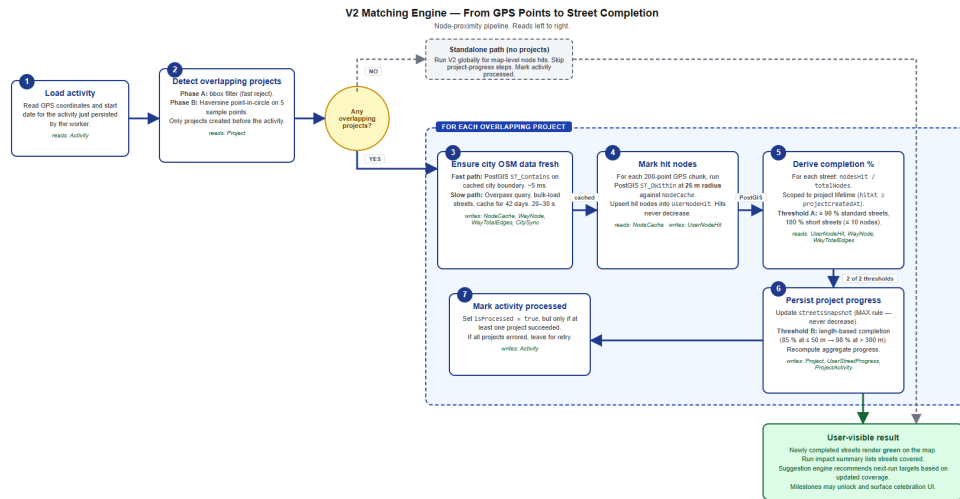


Figure 4.7: V2 node-proximity processing pipeline. GPS coordinates are loaded from the activity record, overlapping projects are detected via a two-phase bounding-box then Haversine check, and ensureCitySynced populates the local NodeCache from Overpass if the city has not been synced within 42 days. The core spatial step uses PostGIS ST_DWithin at a 25 m radius in chunks of 200 GPS points to identify hit nodes, which are upserted into UserNodeHit. Completion is derived per way: streets with more than ten nodes require 90% of nodes hit; shorter streets require 100%. Progress is persisted to the project snapshot using a MAX rule (percentages never decrease) and a secondary length-based completion threshold (85–98% depending on street length).

Overpass dependency) while eliminating the storage constraint that had forced the HMM design into a corner (Newson & Krumm, 2009). Tables from the abandoned intermediate engine iteration (UserEdge, WayCache) remain in the schema as inert evidence of the abandoned path but are no longer written to by the current engine; they appear in the schema diagram (see Figure 4.4 “Street Keeper database schema”, p.69) inside the deprecated_v1_legacy group to make this separation explicit.

The design differences between the two engines across eight properties traced directly from the codebase are summarised (see Table 4.1 “Design comparison of the V1 and V2 matching engines”, p.76).

One observation from this comparison generalises as an insight beyond this project. When consumer smartphone GPS positional error (typically 7–13 m) exceeds the spacing between the discrete spatial units used for completion tracking, a binary hit/miss model absorbs that error at the unit level without propagating it across the street (Merry & Bettinger, 2019; Zandbergen, 2009). A continuous coverage model, by contrast, accumulates positional error along the street’s length, producing unstable percentages that vary across repeated runs of the same route. The V1-to-V2 evolution documented (see section 4.4 “GPS-to-Street Matching Engine”, p.73) was the empirical discovery of this property, and its independent adoption by CityStrides (Chevalier, 2025) suggests that for

Table 4.1: Design comparison of the V1 (polyline coverage) and V2 (node proximity) matching engines.

Property	V1: Polyline Coverage	V2: Node Proximity
Matching model	Each GPS point projected onto nearest polyline; coverage = fraction of length spanned by matched points.	ST_DWithin at 25 m finds nearby NodeCache nodes; each hit recorded as binary row in UserNodeHit.
Completion rule	Length-based thresholds (85–98%); spatial-interval verification when ratio exceeds 1.0.	90% of nodes hit; 100% for short streets (≤ 10 nodes).
External dependency per activity	Overpass API (every run) + Mapbox Map Matching (optional, paid).	None once city synced; local PostGIS spatial join against NodeCache.
Progress storage	UserStreetProgress: one row per user per street; continuous percentage with MAX rule.	UserNodeHit: one row per user per node. Completion derived at query time from hit/total ratio.
Determinism	Non-deterministic: weighted scoring (distance 40%, trajectory 40%, continuity 20%) is order-dependent.	Deterministic: ST_DWithin is a pure spatial predicate; same data always yields same hits.
GPS-drift sensitivity	High: 10 m drift causes parallel streets to become candidates; scoring must disambiguate.	Low: 10 m drift within 25 m radius. Ambiguity only when streets < 50 m apart.
Accuracy (observed)	$\sim 98\%$ with Mapbox snapping; $\sim 85\%$ without.	No aggregate metric: completion derived from individually deterministic node hits.
Data footprint per city	None stored locally; Overpass queried live per activity.	NodeCache + WayNode + WayTotalEdges via on-demand city sync (42-day expiry).

consumer GPS applications the discrete-unit model is the structurally preferable design when deterministic progress tracking is required.

4.4.3 On-Demand City Synchronisation

On-demand city synchronisation bridged the gap between V2's need for cached node data and the storage constraint of the free-tier database. When a user created a project the backend detected the enclosing city via an Overpass `is_in` query, checked the `CitySync` table for a valid cache entry, and if none existed queried Overpass for every street in that city. Results were inserted into `NodeCache`, `WayNode`, and `WayTotalEdges`, with a 42-day expiry (`CITY_SYNC.EXPIRY_DAYS = 42`) chosen to be longer than most realistic editing cycles in a single city but short enough that OSM changes such as new developments would eventually be picked up. Subsequent projects in the same city reused this cached data for free. The design traded first-load latency (tens of seconds for a new city, because the Overpass query and the subsequent inserts both have to complete before the project is usable) for a dramatically smaller database footprint. A known limitation is that the user experience during that first sync is limited to a “Creating...” label on the submit button, with no progress percentage or estimated time; a user creating a project in a large city can wait twenty or thirty seconds with no feedback beyond a spinner. The mitigation is a progressive loading UI that streams sync status over server-sent events, documented as future work (see chapter 7 “Conclusion”, p.114).

The architectural payoff of city synchronisation is that it enables the “no per-activity Overpass dependency” property claimed (see subsection 4.4.2 “Engine V2: Node Proximity”, p.73). Once a city has been synced, a fast-path check avoids Overpass entirely: `findSyncedCityByPoint` runs a PostGIS `ST_Contains` query against the convex hull boundary stored on the `CitySync` row (computed during sync via `ST_ConvexHull` with a ~ 100 m `ST_Buffer` so that streets on the city edge fall inside the polygon). If the point lies inside a non-expired boundary, the function returns the cached `CitySyncRecord` immediately and no Overpass call is made. In practice this means that only the first project creation in a new city triggers an Overpass query; every subsequent project creation, activity processing run, and map-streets request in that city resolves locally against PostGIS. This is the mechanism that converts V2's theoretical independence from Overpass into an operational reality: the V2 pipeline described (see subsection 4.4.2 “Engine V2: Node Proximity”, p.73) calls `ensureCitySynced` at the start of every activity, but for any city that has been synced within the 42-day window that call completes with a single indexed spatial query rather than a network round trip.

A remaining design concern was what happens when multiple activities or project creations target the same unsynchronised city simultaneously, a realistic scenario when a

user first connects their Strava account and the background sync imports dozens of activities at once, all of which may land in the same city. Without protection, each concurrent call would independently detect the city via Overpass and then attempt a full street sync, multiplying both Overpass load and database writes. Three deduplication layers prevent this. First, a grid-cell promise lock (`ensureInProgress`, keyed by a ~ 1 km cell derived from `Math.floor(lat \times 100)`) ensures that concurrent `ensureCitySynced` calls for the same neighbourhood share a single in-flight promise rather than each starting their own detect-and-sync sequence. Second, the `detectCity` function maintains a 60-second result cache (`detectCityCache`) and an in-flight promise map (`detectCityInFlight`) so that the Overpass `is_in` query itself is never duplicated within the same grid cell. Third, a per-`relationId` sync lock (`syncInProgress`) guarantees that only one `syncCity` call per city can be active at any time; concurrent callers wait on the same promise. Beneath all three layers, a serialising throttle (`overpass-throttle.service.ts`) enforces a minimum 2.8-second gap between any two Overpass requests, regardless of caller, to stay within the Overpass public-server rate limits. This layered approach addresses the same connection-pool pressure described (see section 4.1 “System Architecture”, p.65), where worker concurrency was deliberately kept low to prevent background jobs from starving user-facing API requests of database connections.

4.4.4 Suggestion Engine

The suggestion engine recommended areas for the user’s next run based on their street completion data. It operated on project data already loaded in memory and required no additional external API calls.

A simpler alternative would have recommended the nearest single unrun street on each request, without any clustering. This was rejected because an isolated street recommendation provides no spatial context: a user acting on such a suggestion would run one short street and have no natural continuation, since the overhead factor applied to running distance (`CLUSTER_CONNECTING_OVERHEAD = 1.3`) is only meaningful when there is a cluster of nearby streets to connect. Single-street recommendations also offered no “finish what you started” focus, which is the primary motivating signal for users with in-progress streets near the completion threshold.

The engine produced clusters of nearby streets constrained by a distance budget of 2 km of street coverage per cluster (`CLUSTER_MAX_DISTANCE_M = 2000`), a maximum bounding-box diameter of 500 m (`CLUSTER_MAX_BBOX_DIAMETER_M = 500`, which prevents suggestions from snaking across town and becoming unrunnable as a single loop), and a maximum of eight streets per cluster. Up to five clusters were generated per request, sorted by estimated distance so that the quickest suggestion appeared first.

Seed alternation ensured diversity in the output. Even-indexed clusters seeded from